

Section 3: System Design Fundamentals

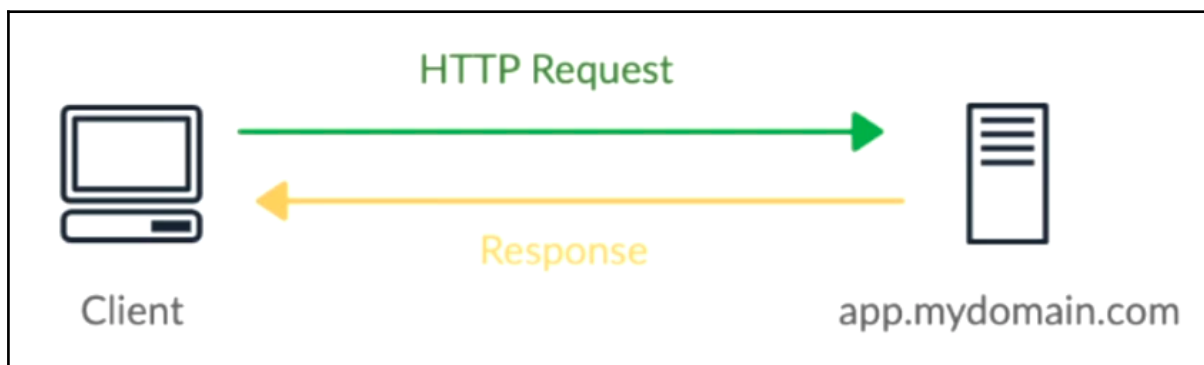
HTTP - The Backbone of the Web

In this session, we are diving into one of the most essential protocols in modern computing which is HTTP or Hyper Text Transfer Protocol. HTTP is the foundation of data communication on the web. Whether you are loading a web page, making an API call, or streaming content, HTTP is working behind the scenes.

Understanding how HTTP operates, its request-response cycle, and its key features like statelessness, methods, status codes, is very crucial for designing scalable and efficient systems. By the end of this session, you will have a solid grasp on how HTTP powers the web and how to use it efficiently in system design.

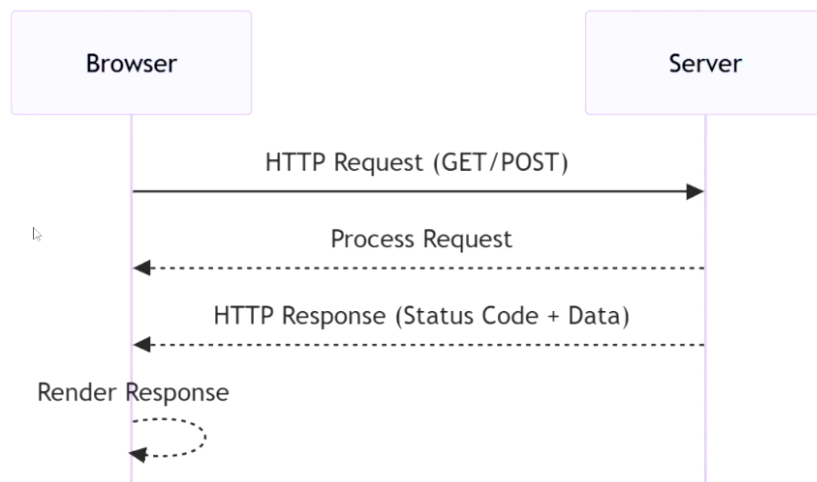
Introduction to HTTP

HTTP stands for HyperText Transfer Protocol and it serves as the foundation of web communication. It defines the rules of how clients like web browsers, mobile apps requests resources and how the server responds with the necessary data. Whether that data is a web page, API response or an image, the response will go back to the client. HTTP operates over TCP protocol and it typically uses port 80 for standard HTTP communication.



HTTP is a text based protocol which means that its messages are human-readable, making it easy to debug. A key characteristic of HTTP is its statelessness, meaning each request is treated independently, without remembering previous interaction. This simplifies the server design but also means some additional mechanism is required to maintain the statefulness if it is required. Mechanisms like cookies, session tokens are sometimes needed for maintaining states. HTTP supports multiple methods that allow getting different types of interaction and data from the server, like GET for retrieving data or POST for submitting information. These core principles make the HTTP the backbone of web communication.

How HTTP Works?



The HTTP follows the client-server model, where the client, like a web browser or a web application, initiates a request, and the server, whether it's a web server or an API server or a database server, processes the request and sends back the response. This simple request-response cycle is the foundation of web communication.

When a client sends an HTTP request, it consists of several key parts or components:

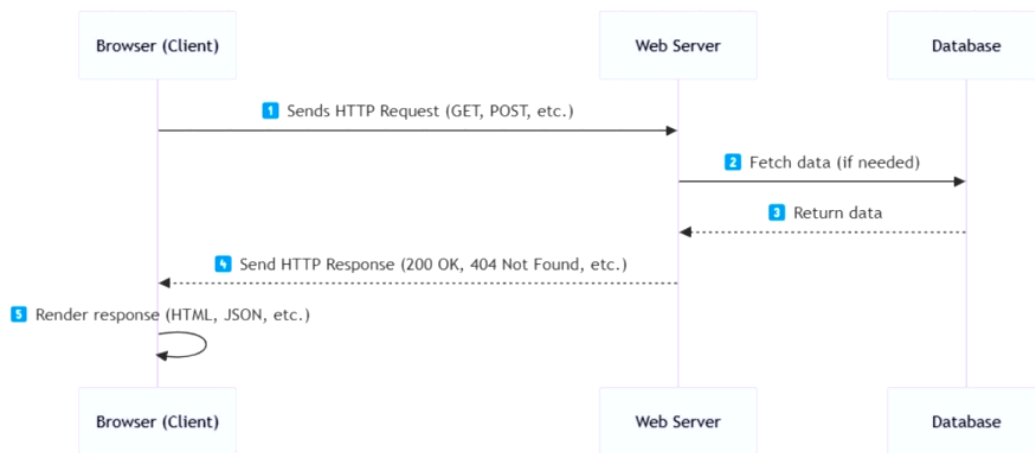
1. **Method:** Method specifies the action that we need to perform, like GET to fetch the data, POST to send the data.
2. **URL:** URL identifies the resource that is being requested from the server.
3. **Header:** Header contains metadata like user agent, authentication tokens, and content type that the client is requesting for.
4. **Body (Optional):** In case you want to send back some data, like in methods like POST or PORT, to send additional information back to the clients, then the body can also be sent from the server to the client.

When the server is sending the request response back to the client, what are its main components?

1. **Status code:** Status code is a numeric code that indicates whether the request is a successful or a failure. 200 is for success, 404 is for not found, and 500 is for some internal server error that has occurred.
2. **Headers:** Headers also come back in response. These headers provide the details about the response, such as content type or some caching information.
3. **Body (Optional):** If the response has a body, the actual content is in the body, like an HTML web page, a JSON response, or an image.

So you can see that this interaction between the client and server happens within milliseconds, and it powers everything from loading a web page to making the web page to making the API call in mobile apps.

The HTTP Request-Response Cycle



HTTP request-response cycle is the core of how web communication works. This process happens every time you visit a website, load an image, or interact with a web app.

- **Step 1:** The browser client sends a request and the web browser actually requests the web server to get some resource. This request includes an HTTP method, whether I want to get something or whether I want to post something. The URL of the resource that is being requested, and a header with additional information like user agent and cookies. It can also send a body in case of PUT and POST if I want to send some data to the web server.
- **Step 2:** The web server will process the request. When the request reaches the web server, it parses the request to understand what the client needs. It checks the authentication and authorization, if at all it is required. It queries the database or retrieves the static content. Then it processes the data and prepares a response to send back.
- **Step 3:** The server generates the response and sends it back. After processing, the server creates an HTTP response, which will contain a status code, stating whether it is successful, failure, or not found. It will have headers, which will have the information about content type and cache control, and a body, which could be an HTML web page, a JSON data, or an image.
- **Step 4:** The browser renders the response. As the client receives the response, if it's an HTML web page, the browser will render it. If it's an API response, the client processes the data and if required, updates the web page, browser or mobile application.

If at all required, the browser can make additional requests, like fetching CSS, JavaScript, or images. All this happens in milliseconds, enabling the fast and seamless web experiences that we rely on everyday.

Stateless Nature of HTTP

Statelessness is the most important characteristic of HTTP. This stateless nature of HTTP has a very big impact on how web applications manage user sessions and data. When we say HTTP is stateless, it means that each request is treated as an independent transaction. The server does not retain memory of the previous request. So if you visit a website, every page request is separate. The server does not remember whether you have been here before or not. Each request must contain all the necessary information for processing that request.

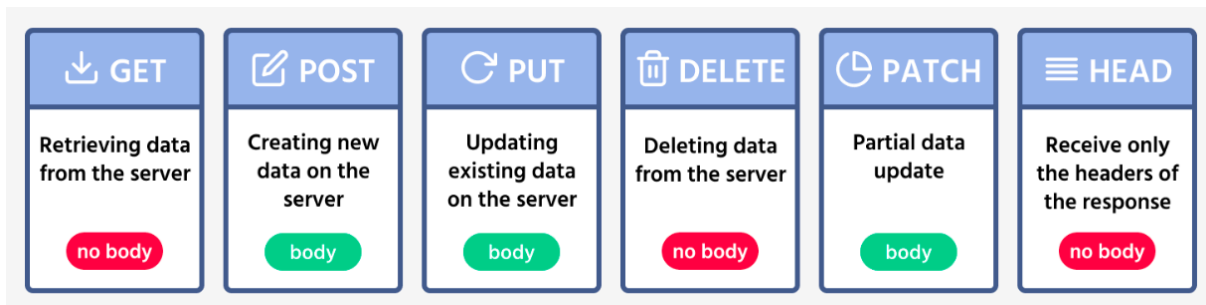
Statelessness makes HTTP simple and scalable. However it also introduces challenges, especially when building interactive web applications. So in HTTP, there is no built-in session management. So the server doesn't know if the same user is making multiple requests. So what it means is that it is very hard to maintain login states. If you log in, the server won't remember whether you have logged in previously or not. So you will need an extra mechanism to be able to do that. Then each request must carry the full context. Clients must include authentication details and other necessary details in every request, so that this server is capable of identifying and knowing that you are a returning user and you have already logged in before.

We can enable sessions and login states in a stateless application protocol using the below techniques:

- **Cookies:** These are small pieces of data stored in the user browser and they are sent with each request and they help to track the user activity. They are commonly used for session management and saving user preferences.
- **Server-Side Sessions:** In this session, the server will store the user data. It will send back a session ID to the client and the client will keep that session ID in the cookie. This session ID will be passed to the server with every user request. And using this ID, the server will identify which user data should be used with this request.
- **Tokens:** There are tokens like JSON web tokens or OAuth tokens which can store the authentication information. So what happens is that tokens are sent with each request to verify user identity. They are widely used in the modern API and authentication systems.

So to summarize, while HTTP itself is stateless, we can use mechanisms like cookies, server-side sessions and tokens to manage state and create a seamless user experience.

HTTP Methods



HTTP Methods are the different ways the clients and servers interact with each other. The below are the HTTP methods:

- **GET:** This is the most commonly used method and it requests data from a server. It can be used to open a web page and it can be used to fetch some data from the API. The best part about GET is GETs are safe and idempotent. Idempotent means repeating the same request again and again won't change anything on the server.
- **POST:** This is used to create a new source on the server. It sends some data to the server to create something new. POSTs are not idempotent, if you send some data again and again to the server, the server will end up creating those many resources again and again.
- **PUT:** This is used to update an existing resource. It will replace some of the properties of an already existing object or resource on the server with whatever the user is trying to send. An example could be if a user wants to update their profile, they can do it. PUTs are idempotent, if we send the same requests multiple times, it should result in the same state. So if you are updating the email ID and sending the new value, if you make a PUT call multiple times, it will end up in the same result.
- **DELETE:** This method removes a resource. It completely removes the resource from the server. DELETES are also idempotent. If you send the same DELETE request multiple times, it will not cause any additional side effects because in the first request, the resource will be deleted. In the second one, it will find that there is nothing to delete, so there are no side effects to it.
- **PATCH:** This method is used to partially update the resource. It is similar to PUT, but it only updates specific fields instead of replacing the complete resource on the server. Not necessarily idempotent meaning the same request could have different effects depending on the previous values that were there.

So each of these methods play a crucial role in web development and API design. Choosing the right method ensures clarity, security and efficiency in communication between clients and servers.

HTTP Status Codes.

Every time you send an HTTP request, the server responds with a status code that tells you whether your request has been successful or something went wrong. The below are some of the status codes based on key categories (100, 200, 300, 400 & 500):

- 1xx - These 100 series are informational messages (Request received, continuing process).
- 2xx - These 200 series will tell you about the success related HTTP codes.
 - 200 OK - Successful Response
 - 201 Created - Resource Successfully Created
- 3xx - These 300 series is for redirection (Further action needed)
 - 301 Moved Permanently - Resource URL changes
 - 304 Not Modified - Use Cached Version
- 4xx - These 400 series id for client errors. So if the client is requesting for a bad resource or the client is not authorized, you will get a 400 series error.
 - 400 Bad Request - Incorrect Request Format
 - 401 Unauthorized - Authentication Required
 - 403 Forbidden - No Permission
 - 404 Not Found - Resource doesn't exist
- 5xx - These 500 series errors are called server errors. These are the errors that come back from the server whenever anything goes wrong.
 - 500 Internal Server Error - Unexpected Server Failure
 - 503 Service Unavailable - Server Overloaded

Understanding these status codes is crucial when debugging issues in web applications and APIs. They help diagnose problems and ensure smooth communication between clients and servers.

What about HTTPS?

So far we have only talked about HTTP and not HTTPS. HTTPS or HyperText Transfer Protocol Secure is simply the secure version of HTTP. It uses SSL encryption to protect the data as it travels between clients and server. So these encryption requires 3 critical security aspects:

1. Confidentiality: Your data is encrypted and preventing the data from hackers is essential nowadays.
2. Integrity: It ensures that the data isn't altered during the transmission.
3. Authentication: It verifies the website's identity is actually genuine and thus attacking the phishing attacks.

Websites that handle sensitive data such as online banking, e-commerce and login pages must use HTTPS to keep users safe. Another key difference is that HTTPS runs on 443 instead of port 80.

So with increasing security concerns, most modern browsers flag HTTP websites as not secured right now. And search engines like google and other search engines actually prioritizes HTTPS for better ranking also. So whenever you are designing web applications, APIs or any system that handles private data, make sure that you are always using HTTPS instead of HTTP.

Summary & Key Takeaways

- HTTP is the foundation of web communication
- It follows a request-response model
- HTTP is stateless, requiring mechanisms like cookies & sessions
- HTTP methods define actions, and status codes indicate responses
- What's Next:
 - REST & RESTfulness - API Design Principles

HTTP Interview Questions

- Basic Questions:
 - What is HTTP, and how does it work?
 - Why is HTTP considered a stateless protocol?
 - What are the key differences between HTTP and HTTPS?
- Intermediate Questions:
 - Explain the HTTP request-response cycle with an example.
 - What are HTTP methods? When would you use PUT vs. PATCH?
 - What are HTTP status codes? Give examples of 2xx, 3xx, 4xx, and 5xx status codes.
- Advanced Questions:
 - How do cookies, sessions, and tokens help maintain state in HTTP?
 - What is the difference between 301 (Moved Permanently) and 302 (Found) redirections?
 - How does caching work in HTTP, and which headers control it?
 - What security risks are associated with HTTP, and how can they be mitigated?